C语言笔记

C语言笔记

目录

1. 第1章 C语言入门基础 —— 程序骨架、注释、常量与变量、printf / scanf
2. 第2章 数据与运算 —— 存储规则、数据类型、运算符、进制输出、类型转换、逻辑运算
3. 第3章 流程控制 —— if、switch、三种循环、break / continue / goto、经典循环应用
4. 第4章 函数 —— 定义与调用、返回值、原型声明、值传递、本地变量
5. 第5章 数组 —— 定义与初始化、遍历、二维数组
6. 第6章 指针 —— 取地址、指针变量、const、指针运算、动态内存分配
7. 第7章 字符串 —— 字符数组与 `\0`、字符串函数
8. 第8章 枚举 —— 常量符号的命名与自动计数
9. 第9章 结构体 —— 复合类型、成员访问、内存对齐
10. 附录 练习题 —— 14 道经典题目思路与参考代码

笔记概要

- **所属课程：** C语言程序设计
- **核心内容：** 从数据类型与运算符出发，依次学习流程控制、函数、数组、指针、字符串、枚举与结构体，建立完整的 C 语言知识体系
- **学习建议：** 每章代码务必亲手敲一遍并观察运行结果；数组与指针是重难点，建议结合内存模型理解
- **关键词：** 数据类型、运算符、循环、函数、数组、指针、字符串、结构体

第1章 C语言入门基础

1.1 第一个 C 程序

```c
#include <stdio.h>

int main()
{
    printf("Hello, world!\n");
    return 0;
}
```

- 每个 C 程序从 `main` 函数开始执行
- 每条语句以分号 `;` 结尾，大括号 `{ }` 表示一个代码块
- `#include <stdio.h>` 引入标准输入输出库

1.2 注释

1. 单行注释：`//注释内容`
2. 多行注释：`/*注释内容*/`
3. VS 快捷键：
    - 添加注释：`Ctrl + K` `Ctrl + C`
    - 取消注释：`Ctrl + K` `Ctrl + U`

提示

注释是写给读代码的人（包括未来的自己）看的，编译器会直接忽略它们。复杂的逻辑一定要写注释。

1.3 关键字

`int`、`return` 等都是 C 语言的关键字（保留字），它们有固定的含义，不能用作变量名。

1.4 常量

类型	说明	例子
整型常量	正整数、负整数、零	`18`、`-5`、`0`
实型常量	所有带小数点的数字	`0.93` 等同于 `.93`，`18.0` 等同于 `18.`
字符常量	单引号引起来的字母、数字、英文符号	`'a'`、`'1'`、`'+'`
字符串常量	双引号引起来的内容	`"hello"`

1.5 变量

语法：`数据类型 变量名;`

```
int a = 10;
int b = 20;
int c = a + b;
```

要点：

- 一条语句可以定义多个变量：`int a, b, c;`（定义了三个变量）
- 变量只能保存一个值；同一作用域内变量名不允许重复定义
- 变量在使用之前一定要赋值（初始化）
- 变量定义的位置：VS（C99 及以后的标准）中任意位置均可定义；VC（C89 标准）要求变量定义在块的最前面，然后才能开始使用
- 变量的作用域范围一定要清楚（详见 4.7 本地变量）

⚠ 易错点

未初始化的变量中存放的是内存里残留的随机值，直接使用会产生不可预料的错误。

1.6 输出 printf

`printf(参数1, 参数2);` —— 参数 1 是输出内容的最终样式（含占位符），参数 2 是填补的内容。

```
printf("%d", 18);   // 以整型格式输出 18
```

占位符	含义
`%d`	整型（int）
`%f`	实型（float / double 输出均可用）
`%lf`	实型（double，主要用于 scanf）
`%c`	字符型（char）
`%s`	字符串
`%zu`	sizeof 测量结果的占位符

1.7 输入 scanf

- scanf 需要配合 `&` 取地址：`scanf("%d", &x);`
- 输入浮点数（double）时必须用 `%lf` 作占位符
- scanf 不会像 printf 那样自动做类型提升（详见 2.9）

1.8 换行符

平台	换行符
Windows	`\r\n`
Mac（早期）	`\r`
Linux	`\n`

> **提示**
>
> C 代码中统一写 `\n` 即可，编译器与运行环境会处理平台差异（现代 macOS 也使用 `\n`）。

第2章 数据与运算

2.1 计算机的存储规则

- 二进制：逢二进一，借一当二（0、1）
- 十进制：逢十进一，借一当十（0 ~ 9）
- 计算机内部所有数据都以二进制形式存储

2.2 数据类型

数据类型决定两件事：变量中能存储什么类型的数据（整数、小数、字符等）；存储空间的大小。

类别	类型
整数	`short`、`int`、`long`（数值后缀 `L`，打印用 `%ld`）、`long long`（C99，后缀 `LL`，打印用 `%lld`）
小数	`float`、`double`
字符	`char`

- 各类型所能表达数的范围（大致）：`char < short < int < float < double`
- 内存中表达的形式：整数用二进制（补码）；浮点数用编码（符号位 + 指数 + 尾数）
- 浮点数表示范围更大，但存在精度限制

> 🕮 **定义**
>
> **sizeof 是静态的**：`sizeof` 的结果在编译期就确定了，运行期不会改变。

2.3 sizeof

- 语法：`sizeof(变量名/数据类型)`，返回所占的字节数
- 用 `%zu` 作占位符输出

```
printf("%zu", sizeof(int));   // 通常输出 4
```

2.4 整数运算与浮点数

- 两个整数的运算结果只能是整数（除法直接截断小数部分）
- 浮点数：带小数点的数值

⚠ **易错点**

`5 / 2` 的结果是 `2` 而不是 `2.5`。要得到小数结果，至少一个操作数应是浮点数：`5.0 / 2`。

2.5 运算符

取余 `%`

- `%` 作为运算符表达的是取余（求余数），如 `7 % 3` 的结果是 `1`
- 只能对整数使用

复合赋值

`+=`、`-=`、`*=`、`/=`、`%=`：如 `a += 5;` 等价于 `a = a + 5;`

递增递减 `++` / `--`

- `a++` 的值是 a 加 1 以前的值；`++a` 的值是 a 加了 1 以后的值；无论哪种写法，a 本身都已经加了 1
- 后缀：加 1 以前的值；前缀：加 1 以后的值
- `count++` 等效于 `count += 1`，等效于 `count = count + 1`

比较运算符

`==` 相等；`!=` 不相等；`>` 大于；`>=` 大于或等于；`<` 小于；`<=` 小于或等于

- 比较结果成立则输出 `1`（真），不成立则输出 `0`（假）
- 判断是否相等（`==`、`!=`）的优先级比判断大小（`<`、`<=`、`>`、`>=`）的优先级低
- 连续的关系运算符从左到右进行

2.6 二进制负数与 unsigned

- 二进制负数：仿照十进制，用一个特殊的标志表示负数；取中间的数为 0，比它小的是负数，比它大的是正数
- 补码：负数以补码形式存储——补码和原码（源码）可以加出一个溢出的 1，两者互为"凑满溢出"的关系
- 直观理解：一个字节（8 位）能表示 -128 ~ 127；在 0 的左边依次是 -1、-2、……

- `unsigned`：不以补码的形式看待数——不能表达负数，但正数的范围扩大一倍

✎ 例题

`unsigned char` 表示 0 ~ 255，而 `signed char` 表示 -128 ~ 127。

2.7 整数与浮点的输入输出进阶

- `%o` 用于输出八进制；`%x` 用于输出十六进制
- `%e` 或 `%E` 以科学计数法输出
- 输出精度：在 `%` 和 `f` 之间加上 `.n` 可以指定输出小数点后 n 位（输出时做四舍五入，n 是要保留的位数），如 `%.2f`
- `inf` 表示超过范围的浮点数（无穷大）；`nan` 表示不存在的浮点数
- 带小数点的字面量是 `double`；`float` 需要用 `f` 或 `F` 后缀表明身份，如 `3.14f`
- `fabs()`：求绝对值（需引入 `math.h`）

💡 提示

选择浮点类型：如果没有特殊需要，选用 `double`。

2.8 字符类型

- `char`：字符的输入、输出与计算都按 ASCII 码进行
- 常用 ASCII 码：`'A'` 是 65，`'a'` 是 97，`'0'` 是 48

逃逸字符

字符	意义	字符	意义
`\b`	回退一格	`\"`	双引号
`\t`	到下一个表格位	`\'`	单引号
`\n`	换行	`\\`	反斜杠本身
`\r`	回车		

2.9 类型转换

- 自动类型转换：对于 printf，任何小于 int 的类型会被转换成 int；float 会被转换成 double
    - 但 scanf 不会自动转换——short 输入要用 `%hd`
- 强制类型转换（通常是转换成较小的类型）：格式 `(类型)值`

```
int i = (int)3.14;   // i == 3
```

- 强制转换类型的优先级高于四则运算

2.10 逻辑类型与运算

```
#include <stdbool.h>   // 之后就可以使用 bool、true、false
```

运算符	描述	示例	结果
`!`	逻辑非	`!a`	a 是 true 则结果为 false；a 是 false 则结果为 true
`&&`	逻辑与	`a && b`	a 和 b 都是 true 结果才为 true，否则为 false
`	`		`	逻辑或

- 逻辑运算是对逻辑量进行的运算，结果只有 0 或 1
- 逻辑量是关系运算或逻辑运算的结果
- 优先级：`!` > `&&` > `||`，例如 `!done && (count > MAX)`

运算符优先级总表：

优先级	运算符	结合性
1	`()`	从左到右
2	`!` `+` `-` `++` `--`（单目）	从右到左
3	`*` `/` `%`	从左到右
4	`+` `-`	从左到右
5	`<` `<=` `>` `>=`	从左到右
6	`==` `!=`	从左到右
7	`&&`	从左到右
8	`	`	
9	`=` `+=` `-=` `*=` `/=` `%=`	从右到左

短路

- 逻辑运算是自左向右进行的，如果左边的结果已经能够决定结果了，就不会做右边的计算
- 对于 `&&`：左边是 false 时就不做右边了
- 对于 `||`：左边是 true 时就不做右边了

> ⚠ **易错点**
>
> 不要把赋值或复合赋值写进逻辑表达式（如 `a==6 && b+=1`）——短路会让右边的赋值"有时执行、有时不执行"。

条件运算符

```
count = (count > 20) ? count - 10 : count + 10;
```

- `?` 前面是条件，`?` 后面是条件满足时的值，`:` 后面是条件不满足时的值
- 与下面的 if-else 完全等价：

```
if ( count > 20 )
    count = count - 10;
else
    count = count + 10;
```

- 条件运算符的优先级只比赋值高，低于其他运算符

2.11 逗号运算

- 逗号用来连接两个表达式，并且以右边表达式的值作为整个表达式的结果
- 逗号的优先级是最低的
- 主要在 for 中使用：

```
for (i = 1, sum = 0; i <= 10; i++) { ... }
```

> 🕮 **区分**

调用函数时的逗号和逗号运算符怎么区分？调用函数时圆括号里的逗号是标点符号，不是运算。`f(a, b)` 传两个值；`f((a, b))` 是逗号运算，只传一个值（b 的值）。

第3章 流程控制

3.1 if 条件语句

格式：`if (条件) {代码}`

```
if (score >= 60) {
    printf("及格\n");
}
```

3.2 if-else 语句

格式：`if (条件) {代码} else {代码}`

```
if (score >= 60) {
    printf("及格\n");
} else {
    printf("不及格\n");
}
```

- if-else 可以嵌套（else 与最近的、没有 else 配对的 if 配对）
- 多分支用 `else if`：

```
if (条件1) { ... }
else if (条件2) { ... }
else if (条件3) { ... }
else { ... }
```

3.3 switch case 语句

```
switch (控制表达式) {
case 常量:
    语句;
    break;
case 常量:
    语句;
    break;
```

```
default:
    语句
}
```

- 控制表达式必须是 int 型的
- `case` 后面的常量可以是常数，也可以是常数计算表达式
- `break` 是这一 case 的终止信号，如果没有 break 则会"穿透"进入下一 case
- `default` 匹配所有未命中的情况

3.4 while 循环

格式：`while (条件) {循环体}`

```
int i = 1;
while (i <= 10) {
    printf("%d ", i);
    i++;
}
```

- if 和 while 的区别：if 是一次性判断，while 是多次循环

3.5 do-while 循环

```
do {
    <循环体语句>
} while (<循环条件>);
```

- 和 while 的区别：do-while 是先进循环体再判断条件（至少执行一次）；while 是先判断条件才进循环体

3.6 for 循环

格式：`for (初始动作; 条件; 每轮的动作) {循环体}`

```
for (i = 1; i <= 10; i++) {
    sum += i;
}
```

- for 中的每一个表达式都是可以省略的
- `for (; 条件; )` 等价于 `while (条件)`（注意：分号不能省）

3.7 三种循环的选择

场景	选择
有固定次数	for
必须执行一次	do-while
其他情况	while

3.8 break 与 continue

- `break`：跳出（它所在的）循环
- `continue`：跳过循环这一轮剩下的语句，进入下一轮循环
- break 和 continue 只能对它所在的那层循环起作用

3.9 嵌套循环与 goto

- 循环可以嵌套：内层循环完整走完一遍，外层循环才走一步
- `goto` 可以跳到所指标号处（如 `goto out`），常用于多重循环中一次性跳出：

```
for (...) {
    for (...) {
        if (...) {
            goto out;
        }
    }
}
out:
    return 0;
```

> **提示**
>
> goto 只建议用于"多重循环整体跳出"这类场景，滥用会让程序流程混乱难读。

3.10 随机数 rand()

- `rand()` 返回一个 0 到 `RAND_MAX` 之间的随机整数（需引入 `stdlib.h`）
- 配合 `%` 可以得到指定范围的随机数：`rand() % 100` 得到 0 ~ 99
- 默认每次运行序列相同，可配合 `srand(time(0))` 改变随机种子

3.11 经典循环应用

整数的分解

- 取个位数：对整数做 `%10` 的操作
- 去掉个位数：对整数做 `/10` 的操作
- 取十位数：先去掉个位数（`/10`），再对得到的结果做 `%10` 的操作
- 由此类推，可以得到任意一位

取逆序

```c
#include <stdio.h>

int main()
{
    int x;
    // scanf("%d", &x);
    x = 12345;
    int digit;
    int ret = 0;

    while (x > 0) {
        digit = x % 10;
        ret = ret * 10 + digit;
        printf("x=%d, digit=%d, ret=%d\n", x, digit, ret);
        x /= 10;
    }
    printf("%d", ret);

    return 0;
}
```

> **提示**
>
> 核心是两句话：`digit = x % 10;` 取出最后一位，`ret = ret * 10 + digit;` 把这一位接到结果尾部。中间那行 printf 是调试输出，用来观察每一步的变化。

阶乘

```c
int n;
scanf("%d", &n);
int fact = 1;
int i = 1;
while (i <= n) {
```

```
    fact *= i;
    i++;
}
printf("%d!=%d\n", n, fact);
```

求和

```
int i;
int sum;
for (i = 1, sum = 0; i <= 10; i++) {
    sum += i;
}
printf("%d到%d的和是%d\n", 1, 10, sum);
```

> **提示**
>
> 循环里经常要做"计数"和"累加"两件事：计数用 `count = 0; count++;`，累加用 `sum = 0; sum += x;`，注意初始值一定是 0。

第4章 函数

4.1 为什么要写函数

同样的代码写三遍，既啰嗦又难改；把重复代码提取成一个函数，一处定义、多处调用：

```
int i;
int sum;
for (i=1, sum=0; i<=10; i++) {
    sum += i;
}
printf("%d到%d的和是%d\n", 1, 10, sum);
for (i=20, sum=0; i<=30; i++) {
    sum += i;
}
printf("%d到%d的和是%d\n", 20, 30, sum);
for (i=35, sum=0; i<=45; i++) {
    sum += i;
}
printf("%d到%d的和是%d\n", 35, 45, sum);
```

提取成函数后：

```c
void sum(int begin, int end)
{
    int i;
    int sum = 0;
    for (i=begin; i<=end; i++) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}

int main()
{
    sum(1, 10);
    sum(20, 30);
    sum(35, 45);
    return 0;
}
```

4.2 函数的定义

函数由**函数头**（第一行）和**函数体**（`{ }` 内的代码）组成：

```c
void sum(int begin, int end)   // ← 函数头（函数原型行）
{                              // ┐
    int i;                     // │
    int sum = 0;               // │ 函数体
    for (i=begin; i<=end; i++) {   // │
        sum += i;              // │
    }                          // │
    printf("%d到%d的和是%d\n", begin, end, sum);   // │
}                              // ┘
```

函数头的各部件：

部件	上面的例子中
返回类型	`void`（不返回任何东西）
函数名	`sum`
参数表	`(int begin, int end)`

4.3 调用函数

格式：`函数名(参数值);`

- 即使没有参数也要写 `()`

- 如果有参数，则需要给出正确的数量和顺序（类型也要正确）
- 可以传递给函数的值是"表达式的结果"，包括：字面量、变量、函数返回值、计算结果

```
int a, b, c;
a = 5;
b = 6;
c = max(10, 12);          // 字面量
c = max(a, b);            // 变量
c = max(c, 23);           // 变量 + 字面量
c = max(max(23, 45), a);     // 函数的返回值
c = max(23+45, b);        // 计算结果
```

4.4 返回值

```
int max(int a, int b)
{
    int ret;
    if (a > b) {
        ret = a;
    } else {
        ret = b;
    }
    return ret;
}
```

- `return` 停止函数的执行，并送回一个值
- 两种形式：`return;` 和 `return 表达式;`
- 一个函数里可以出现多个 return 语句

没有返回值的函数（void）：

- `void 函数名(参数表)`
- 不能使用带值的 return；可以没有 return
- 调用的时候不能把返回值拿来参与运算
- 如果函数有返回值，则必须使用带值的 return

4.5 函数原型声明

- C 的编译器自上而下顺序分析你的代码

- 在调用 `sum(1, 10)` 的时候，编译器需要知道 `sum()` 的样子：要几个参数、每个参数的类型如何、返回什么类型
- 函数头以分号 `;` 结尾，就构成了函数的**原型**，原型的目的就是提前告诉编译器这个函数长什么样

```
double max(double a, double b);   // 函数原型（提前声明）

int main()
{
    int a, b, c;
    a = 5;
    b = 6;
    c = max(10, 12);
    printf("%d\n", c);
    return 0;
}

double max(double a, double b)    // 实际的函数定义可以写在后面
{
    ...
}
```

- 旧标准习惯把函数原型写在调用它的函数里面；现在一般写在调用它的函数前面

4.6 值传递（传值）

```
void swap(int a, int b);

int main()
{
    int a = 5;
    int b = 6;
    swap(a, b);
    printf("a=%d b=%d\n", a, b);
    return 0;
}

void swap(int a, int b)
{
    int t = a;
    a = b;
    b = t;
}
```

🏛 核心结论

C 语言在调用函数时，永远只能传值给函数。 函数里对参数的修改不会影响调用处的变量——所以上面这样的代码**不能**交换 a 和 b 的值。想交换，必须用指针（见 6.4）。

- 每个函数有自己的变量空间，参数也位于这个独立的空间中，和其他函数没有关系
- 过去把函数参数表中的参数叫做"形式参数"，调用函数时给的值叫做"实际参数"。这种称呼容易让初学者误以为调用函数时把变量（而不是值）传了进去，所以现在不建议继续使用

4.7 本地变量

- **生存期**：什么时候这个变量开始出现了，到什么时候它消亡了
- **作用域**：在（代码）什么范围内可以访问这个变量
- 对于本地变量，这两个问题的答案是统一的：**大括号内——块**

本地变量的规则：

- 本地变量定义在块内——可以是函数的块内，也可以是语句的块内，甚至可以随便拉一对大括号来定义变量
- 程序运行进入这个块之前，其中的变量不存在；离开这个块，其中的变量就消失了
- 块外面定义的变量在里面仍然有效
- 块里面定义了和外面同名的变量，则掩盖了外面的
- 不能在一个块内定义同名的变量
- 本地变量不会被默认初始化
- 参数在进入函数的时候被初始化了

没有参数时：

- `void f(void);` —— 明确表示没有参数
- `void f();` —— 在传统 C 中，它表示 f 的参数表未知，并不表示没有参数

⚠ 易错点

C 语言不允许函数嵌套定义（函数里不能再定义函数）。

第5章 数组

5.1 定义数组

```
<类型> 变量名称[元素数量];

int grades[100];
double weight[20];
```

- 元素数量必须是整数
- C99 之前：元素数量必须是编译时刻确定的字面量

5.2 数组的特点

- 数组是一种容器（放东西的东西），特点是：
    - 其中所有的元素具有相同的数据类型
    - 一旦创建，不能改变大小
    - 数组中的元素在内存中是连续依次排列的
- 数组的每个单元就是该类型的一个变量，可以出现在赋值的左边或右边（在赋值左边的叫做**左值**）：

```
int a[10];              // 10 个单元：a[0], a[1], ... a[9]
a[2] = a[1] + 6;        // 下标做左值，也可以做右值
```

- 使用数组时放在 `[]` 中的数字叫做下标或索引，**下标从 0 开始计数**：`grades[0]`、`grades[99]`、`average[5]`

5.3 下标越界

⚠ 易错点

编译器和运行环境都**不会检查**数组下标是否越界（无论读还是写都不检查）。一旦程序运行，越界的数组访问可能造成问题、导致程序崩溃（segmentation fault），但也可能运气好没造成严重后果——所以保证下标在 `[0, 数组的大小-1]` 是程序员的责任。

- `int a[0];` 可以存在，但是无用

5.4 数组初始化

- 集成初始化：`int a[10] = {0};`（其余位置自动补零）
- C99 支持定位初始化（C99 ONLY）：

```
int a[10] = {
    [0]=2, [2]=3, 6,
};
```

- 用 `[n]` 在初始化数据中给出定位
- 没有定位的数据接在前面的位置后面（上面 6 落在 a[3]）
- 其他位置的值补零
- 也可以不给出数组大小，让编译器算
- 特别适合初始数据稀疏的数组
- 数组的大小：`sizeof` 给出数组所占的字节数（元素个数 = `sizeof(a) / sizeof(a[0])`）
- 数组的赋值：一个数组不能直接赋值给另一个数组（只能逐个单元复制）

const 数组：

```
const int a[] = {1,2,3,4,5,6,};
```

- 数组变量已经是 const 的指针了，这里的 const 表明数组的每个单元都是 `const int`
- 所以必须通过初始化进行赋值

5.5 遍历数组

通常都是使用 for 循环，让循环变量 i 从 0 到 `<数组的长度`，这样循环体内最大的 i 正好是数组最大的有效下标：

```
for(i=0; i<length; i++){
    b[i] = a[i];            // 复制数组
}
```

```
for(i=0; i<length; i++){
    if(a[i] == key){        // 查找
        ret = i;
        break;
```

```
    }
}
```

```
for(i=0; i<number; i++){
    if(number[i] > average){
        printf("%d ", number[i]);
    }
}
```

```
for(i=0; i<number; i++){
    count[i] = 0;           // 清零
}
```

⚠ 易错点

常见错误：① 循环结束条件是 `<=数组长度`（越界）；② 离开循环后，继续用 i 的值来做数组元素的下标（此时 i 已经等于数组长度）。

完整例子：统计数字出现次数

```
const int number = 10;      // 数组的大小
int x;
int count[number];          // 定义数组
int i;
for(i=0; i<number; i++){    // 初始化数组
    count[i] = 0;
}
scanf("%d", &x);
while(x != -1){
    if(x>=0 && x<=9){
        count[x]++;         // 数组参与运算
    }
    scanf("%d", &x);
}
for(i=0; i<number; i++){    // 遍历数组输出
    printf("%d:%d\n", i, count[i]);
}
```

5.6 二维数组

```
int a[3][5];   // 三行五列的矩阵
```

- `a[i, j]` 是错误的——逗号运算让整个表达式最后只表达为 `a[j]`（详见 2.11），二维数组必须写 `a[i][j]`
- 初始化：

```
int a[][5] = {
    {0,1,2,3,4},
    {2,3,4,5,6},
};
```

- 列数是必须给出的，行数可以由编译器来数
- 每行一个 `{}`，逗号分隔
- 最后的逗号可以存在，有古老的传统
- 如果省略，表示补零
- 也可以用定位（C99 ONLY）

第6章 指针

6.1 取地址运算 `&`

- `scanf("%d", &i);` 里的 `&`：获得变量的地址，它的操作数必须是变量
- `&` 不能对其他东西取地址（如 `a++`、`++i` 不是变量）
- 想要输出地址，用 `%p` 当占位符：

```
int i;
printf("%p", &i);
```

- 地址的大小是否与 int 相同取决于编译器
- 相邻数组元素的地址差正好是元素类型的大小（int 数组相邻元素相差 4 个字节）

6.2 指针变量

- 指针就是保存地址的变量

```
int i;
int* p = &i;
```

⚠ **易错点**

`int* p, q;` 中**只有 p 是指针，q 是普通的 int**！星号只修饰最近的变量名（`int *p, q;` 与之等价）。建议每行只定义一个指针。

- 示意图：

```
普通变量的值是实际的值：
count ┌─────┐
      │  7  │
      └─────┘

指针变量的值是另一个变量的地址：
countPtr ──────► count
```

- 指针变量的值是"具有实际值的变量的地址"

6.3 解引用 `*`

- `*` 是一个单目运算符，用来访问指针的值所表示的地址上的变量
- 可以做右值也可以做左值（左边是写入，右边是读取）：

```
int k = *p;     // 读取：k 得到 p 所指变量的值
*p = k + 1;     // 写入：p 所指的变量被改成 k+1
```

- `*p` 是一个 int

6.4 指针的应用场景

1. 交换两个变量的值（配合函数，实现真正的交换）

```
void swap(int *pa, int *pb)
{
    int t = *pa;
    *pa = *pb;
    *pb = t;
}
```

💡 **提示**

对比 4.6 的失败版本：函数拿到地址后，通过 `*` 访问到的就是调用处真正的变量，所以能修改它们。

2. 函数返回多个值：函数返回运算的状态，结果通过指针返回

- 常用的套路是让函数返回特殊的不属于有效范围内的值来表示出错：`-1` 或 `0`（在文件操作中会看到大量例子）
- 但是当任何数值都是有效的可能结果时，就得"状态"和"结果"分开返回了

3. 其他典型场景：

- 需要传入较大的数据时用作参数（省去复制）
- 传入数组后对数组做操作
- 函数返回不止一个结果
- 需要用函数来修改不止一个变量
- 动态申请的内存

6.5 const 与指针

指针是 const：表示一旦得到了某个变量的地址，不能再指向其他变量

```
int* const q = &i;
*q = 26;     // OK
q++;         // ERROR
```

所指是 const：表示不能通过这个指针去修改那个变量（并不能使得那个变量成为 const）

```
const int *p = &i;
*p = 26;     // ERROR (*p 是 const)
i = 26;      // OK
p = &j;      // OK
```

> 🏛 **核心结论**
>
> 规律：**const 在 `*` 的前面 → 所指的东西不能修改；const 在 `*` 的后面 → 指针本身不能被修改。**

转换：总是可以把一个非 const 的值转换成 const 的

```
void f(const int* x);
int a = 15;
f(&a);           // OK
const int b = a;
f(&b);           // OK
b = a + 1;       // Error!
```

- 当要传递的参数的类型比地址大的时候，这是常用的手段：既能用比较少字节数传递值给参数，又能避免函数对外面的变量的修改

保护数组值：因为把数组传入函数时传递的是地址，所以那个函数内部可以修改数组的值。为了保护数组不被函数破坏，可以设置参数为 const：

```
int sum(const int a[], int length);
```

6.6 指针运算

- 给一个指针加 1 表示要让指针指向下一个变量（地址增加 `sizeof(类型)`）：

```
int a[10];
int *p = a;
*(p+1)  // -> a[1]
```

- 如果指针不是指向一片连续分配的空间（如数组），则这种运算没有意义
- 可以对指针做的算术运算：
    - 给指针加、减一个整数（`+`、`+=`、`-`、`-=`）
    - 递增递减（`++` / `--`）
    - 两个指针相减（得到它们之间相差的元素个数）

`*p++`：取出 p 所指的那个数据来，完事之后顺便把 p 移到下一个位置去

- `*` 的优先级虽然高，但是没有 `++` 高
- 常用于数组类的连续空间操作
- 在某些 CPU 上，这可以直接被翻译成一条汇编指令

指针比较：`<`、`<=`、`==`、`>`、`>=`、`!=` 都可以对指针做，比较的是它们在内存中的地址；数组中的单元的地址肯定是线性递增的

0 地址：

- 内存中有 0 地址，但 0 地址通常是个不能随便碰的地址，所以指针不应该具有 0 值
- 因此可以用 0 地址来表示特殊的事情：返回的指针是无效的；指针没有被真正初始化（先初始化为 0）
- `NULL` 是一个预定义的符号，表示 0 地址

指针的类型：

- 无论指向什么类型，所有指针的大小都是一样的，因为都是地址
- 但是指向不同类型的指针是不能直接互相赋值的，这是为了避免用错指针
- `void*` 表示"不知道指向什么的指针"；计算时与 `char*` 相同（但不相通）
- 指针也可以转换类型：

```
int *p = &i;
void *q = (void*)p;
```

- 这并没有改变 p 所指的变量的类型，而是让后人用不同的眼光通过 q 看它所指的变量

6.7 数组与指针

- 数组变量本身表达地址，所以：

```
int a[10];
int *p = a;    // 无需用 & 取地址
```

- 但是数组的单元表达的是变量，需要用 `&` 取地址
- `a == &a[0]`
- `[]` 运算符可以对数组做，也可以对指针做：`p[0]` <==> `a[0]`
- `*` 运算符可以对指针做，也可以对数组做：`*a = 25;`
- 数组变量是 const 的指针，所以不能被赋值：`int a[]` <==> `int * const a = ...`

传入函数的数组成了什么?

```
int isPrime(int x, int knownPrimes[], int numberofKnownPrimes)
{
    int ret = 1;
```

```
    int i;
    for(i=0; i<numberofKnownPrimes; i++){
        if(x % knownPrimes[i] == 0){
            ret = 0;
            break;
        }
    }
    return ret;
}
```

- 函数参数表中的数组实际上是指针：`sizeof(a) == sizeof(int*)`
- 但是可以用数组的运算符 `[]` 进行运算

以下四种函数原型是等价的：

```
int sum(int *ar, int n);
int sum(int *, int);
int sum(int ar[], int n);
int sum(int [], int);
```

6.8 动态内存分配

```
#include <stdlib.h>
void* malloc(size_t size);
```

- 向 malloc 申请空间的大小是以字节为单位的
- 返回的结果是 `void*`，需要类型转换为自己需要的类型：

```
(int*)malloc(n * sizeof(int))
```

- 如果没空间，则 malloc 会返回 0 或者 NULL

free()：把申请得来的空间还给"系统"

- 申请过的空间，最终都应该要还（借出来，迟早都是要还）
- 只能还申请来的空间的首地址

⚠ 易错点

常见问题：

1. 申请了没 free —— 长时间运行内存逐渐下降（新手：忘了；老手：找不到合适的 free 时机）
2. free 过了再 free
3. 地址变过了，直接去 free

第7章 字符串

7.1 字符数组与字符串

```
char word[] = {'H','e','l','l','o','!'};
```

word[0]	H	word[3]	l
word[1]	e	word[4]	o
word[2]	l	word[5]	!

⚠ 易错点

这**不是** C 语言的字符串，因为不能用字符串的方式做计算——它缺少结尾的 0。

- 字符串：**以 0（整数 0）结尾的一串字符**
- `0` 或 `'\0'` 是一样的，但是和 `'0'`（字符 0）不同
- 0 标志字符串的结束，但它不是字符串的一部分；计算字符串长度的时候不包含这个 0
- `'\0'` 是一个字节，整数 `0` 是四个字节
- 字符串以数组的形式存在，以数组或指针的形式访问（更多的是以指针的形式）
- `string.h` 里有很多处理字符串的函数
- C 语言的字符串是以字符数组的形态存在的
- 不能用运算符对字符串做运算
- 通过数组的方式可以遍历字符串
- 唯一特殊的地方是字符串字面量可以用来初始化字符数组
- 以及标准库提供了一系列字符串函数

7.2 字符串常量与指针

```
char* s = "Hello, world!";
```

- s 是一个指针，初始化为指向一个字符串常量
- 由于这个常量所在的地方（只读区域），所以实际上 s 是 `const char* s`，但是由于历史的原因，编译器接受不带 const 的写法
- 但是试图对 s 所指的字符串做写入会导致严重的后果
- 如果需要修改字符串，应该用数组：

```
char s[] = "Hello, world!";
```

指针还是数组?

```
char *str = "Hello";
char word[] = "Hello";
```

- 数组：这个字符串"在这里"（作为本地变量空间，自动被回收）
- 指针：这个字符串"不知道在哪里"（处理参数、动态分配空间）
- 如果要**构造**一个字符串 → 数组；如果要**处理**一个字符串 → 指针

7.3 字符串的输入输出

```
char string[8];
scanf("%s", string);
printf("%s", string);
```

- scanf 读入一个单词（到空格、tab 或回车为止）
- scanf 是不安全的，因为不知道要读入的内容的长度（可能越界）

安全的输入：

```
char string[8];
scanf("%7s", string);
```

- 在 `%` 和 `s` 之间的数字表示最多允许读入的字符的数量，这个数字应该比数组的大小小一（给结尾的 `'\0'` 留位置）

⚠ **易错点**

`char *string; scanf("%s", string);` —— 以为 char* 是字符串类型，定义了一个字符串类型的变量 string 就可以直接使用了。由于没有对 string 初始化为 0（没有指向任何有效空间），所以不一定每次运行都出错，但随时可能崩。

空字符串：

```
char buffer[100]="";
```

- 这是一个空的字符串，`buffer[0] == '\0'`

```
char buffer[] = "";
```

- 这个数组的长度只有 1！（只放得下结尾的 `'\0'`）

7.4 程序参数

```
int main(int argc, char const *argv[])
```

- `argv[0]` 是命令本身
- 当使用 Unix 的符号链接时，反映符号链接的名字

7.5 单字符输入输出

- `int putchar(int c);` —— 向标准输出写一个字符；返回写了几个字符，EOF（-1）表示写失败
- `int getchar(void);` —— 从标准输入读入一个字符；返回类型是 int 是为了返回 EOF（-1）
- 输入结束标志：Windows → Ctrl-Z；Unix → Ctrl-D

7.6 字符串函数（string.h）

strlen —— 求字符串长度：

```
size_t strlen(const char *s);
```

- 返回 s 的字符串长度（不包括结尾的 0）

strcmp —— 比较两个字符串：

```
int strcmp(const char *s1, const char *s2);
```

- 返回：`0` 表示 s1 == s2；`1` 表示 s1 > s2；`-1` 表示 s1 < s2（教材简化说法）

> 📖 **补充**
>
> 标准实际规定返回两个字符 ASCII 的差值（正、负或 0），判断时只需知道"是否为 0"，或用 `> 0`、`< 0` 判断大小。

strcpy —— 拷贝字符串：

```
char * strcpy(char *restrict dst, const char *restrict src);
```

- 把 src 的字符串拷贝到 dst（**目的在前，源在后**）
- `restrict` 表明 src 和 dst 不重叠（C99）
- 返回 dst，为了能链起代码来
- 类似地还有 `strcat`（字符串拼接），同样是目的在前、源在后

复制一个字符串（先申请空间，再拷贝）：

```
char *dst = (char*)malloc(strlen(src)+1);
strcpy(dst, src);
```

> 💡 **提示**
>
> 申请 `strlen(src)+1` 个字节而不是 `strlen(src)`——多出的 1 个字节放结尾的 `'\0'`。

字符串中找字符：

```
char * strchr(const char *s, int c);    // 从前往后找
char * strrchr(const char *s, int c);   // 从后往前找
```

- 返回 NULL 表示没有找到

字符串中找字符串：

```
char * strstr(const char *s1, const char *s2);
char * strcasestr(const char *s1, const char *s2);   // 忽略大小写，不区分大小写
```

第8章 枚举

8.1 枚举的定义

枚举是一种用户定义的数据类型，它用关键字 `enum` 以如下语法来声明：

```
enum 枚举类型名字 {名字0, …, 名字n};
```

- 枚举类型名字通常并不真的使用，要用的是在大括号里的名字，因为它们就是常量符号，它们的类型是 int，值则依次从 0 到 n：

```
enum colors {red, yellow, green};
```

- 就创建了三个常量，red 的值是 0，yellow 是 1，而 green 是 2
- 当需要一些可以排列起来的常量值时，定义枚举的意义就是给了这些常量值名字

8.2 枚举的使用

```
#include <stdio.h>

enum color { red, yellow, green };

void f(enum color c);

int main(void)
{
    enum color t = red;
    scanf("%d", &t);
    f(t);
    return 0;
}

void f(enum color c)
{
```

```
    printf("%d\n", c);
}
```

- 枚举量可以作为值
- 枚举类型可以跟上 enum 作为类型
- 但是实际上是以整数来做内部计算和外部输入输出

8.3 套路：自动计数的枚举

```
enum COLOR {RED, YELLOW, GREEN, NumCOLORS};
```

- 最后一个名字 `NumCOLORS` 自动等于枚举量的个数（这里是 3）
- 这样需要遍历所有的枚举量、或者需要建立一个用枚举量做下标的数组的时候就很方便了：

```
#include <stdio.h>

enum COLOR {RED, YELLOW, GREEN, NumCOLORS};

int main(int argc, char const *argv[])
{
    int color = -1;
    char *ColorNames[NumCOLORS] = {
        "red", "yellow", "green",
    };
    char *colorName = NULL;
    printf("输入你喜欢的颜色的代码: ");
    scanf("%d", &color);
    if ( color >=0 && color < NumCOLORS ) {
        colorName = ColorNames[color];
    } else {
        colorName = "unknown";
    }
    printf("你喜欢的颜色是%s\n", colorName);
    return 0;
}
```

8.4 枚举量指定值

- 声明枚举量时候可以指定值：

```
enum COLOR {RED=1, YELLOW, GREEN=5, NumCOLORS};
```

- 未指定值的名字是前一个值 + 1（所以 YELLOW 是 2）

```c
#include <stdio.h>

enum COLOR {RED=1, YELLOW, GREEN=5, NumCOLORS};

int main(int argc, char const *argv[])
{
    printf("code for GREEN is %d\n", GREEN);
    return 0;
}
```

> ⚠ **易错点**
>
> 枚举只是 int：即使给枚举类型的变量赋不存在的整数值也没有任何 warning 或 error。

8.5 小结

- 虽然枚举类型可以当作类型使用，但是实际上很少（需要这么）用
- 如果有意义上排比的名字，用枚举比 const int 方便
- 枚举比宏（macro）好，因为枚举有 int 类型

第9章 结构体

9.1 结构体的定义

结构体是用户自定义的复合数据类型，把相关联的数据组合在一起：

```c
struct date {
    int year;
    int month;
    int day;
};
```

- **匿名结构体**：定义时不写名字，仅当场定义变量（一次性使用）
- **typedef** 起别名，简化使用：

```c
typedef struct {
    int year;
    int month;
```

```
    int day;
} Date;
```

- 之后就可以直接写 `Date today;`

9.2 结构体变量

- **结构体变量赋值**：同类型的结构体变量可以直接整体赋值
- **结构体嵌套**：结构体的成员可以是另一个结构体
- **访问结构体成员**：用 `变量名 + .` 来操作

```
Date today;
today.year = 2026;
today.month = 11;
```

- **如果是指针**的话要用 `变量名 + ->` 来操作：

```
Date *p = &today;
p->day = 4;          // 等价于 (*p).day = 4;
```

9.3 结构体与函数、数组、指针

- **结构体作为函数参数**：整体传值（会复制整个结构体）；结构体大的时候用指针传参更高效
- **结构体数组**：

```
Date dates[10];
dates[0].year = 2020;
```

- **结构体指针**：函数内通过 `->` 修改调用处的结构体：

```
void set_date(Date *p, int y, int m, int d)
{
    p->year = y;
    p->month = m;
    p->day = d;
}
```

9.4 内存对齐

- 结构体在内存中的大小**不一定等于各成员大小之和**
- 编译器为了让访问速度更快，会让每个成员对齐到其类型大小的整数倍地址上（填充 padding 字节）

例题

`struct { char c; int i; };` 的大小是 8 而不是 5——char 之后填充了 3 个字节，让 int 对齐到 4 的整数倍地址。

提示

成员的排列顺序会影响结构体大小：把大的成员放前面通常更省空间。可以用 `sizeof` 查看实际大小。

附录 练习题

题号	题目	主要知识点
1	找零计算器	while 循环、`/` 和 `%`
2	输出一百内的素数	嵌套循环、break
3	输出前 50 个素数	计数器 + while
4	凑硬币	穷举（多重循环）
5	求和	for 循环累加
6	整数的逆序输出	整数分解（%10、/10）
7	整数正序输出	整数分解 + mask
8	求最大公约数	辗转相除法
9	水仙花数	整数分解
10	99 乘法表	嵌套循环
11	统计素数并求和	素数判断 + 计数器
12	猜数字游戏（让计算机猜）	二分查找
13	构造素数表	筛法（数组）
14	井字棋	二维数组、函数

1. 找零计算器

思路： 输入金额，按面额从大到小用贪心法计算各面额需要的张数（`/` 算张数、`%` 算余额）。

```c
#include <stdio.h>

int main()
{
    int money, n, i;
    int faces[] = {100, 50, 20, 10, 5, 1};
    printf("请输入金额：");
    scanf("%d", &money);
    for (i = 0; i < 6; i++) {
        n = money / faces[i];
        money %= faces[i];
        if (n > 0) {
            printf("%d 元：%d 张\n", faces[i], n);
        }
    }
    return 0;
}
```

2. 输出一百内的素数

思路： 素数只能被 1 和自身整除。用嵌套循环逐个判断 2~99 的每个数。

```c
#include <stdio.h>

int main()
{
    int i, j, isPrime;
    for (i = 2; i < 100; i++) {
        isPrime = 1;
        for (j = 2; j < i; j++) {
            if (i % j == 0) {
                isPrime = 0;
                break;
            }
        }
        if (isPrime) {
            printf("%d ", i);
        }
    }
    return 0;
}
```

3. 输出前 50 个素数

思路： 用计数器统计已找到的素数个数，找满 50 个就停止。

```c
#include <stdio.h>

int main()
{
    int x = 2, cnt = 0, i, isPrime;
    while (cnt < 50) {
        isPrime = 1;
        for (i = 2; i < x; i++) {
            if (x % i == 0) {
                isPrime = 0;
                break;
            }
        }
        if (isPrime) {
            printf("%d ", x);
            cnt++;
        }
        x++;
    }
    return 0;
}
```

4. 凑硬币（一角、两角、五角凑出 10 元以下的金额）

思路： 穷举五角和两角的个数，一角的个数由余额推算（单位统一为角，金额 < 100 角）。

```c
#include <stdio.h>

int main()
{
    int amount;              // 单位：角
    int x, y, z, count = 0;
    printf("请输入金额（角，10元以下）：");
    scanf("%d", &amount);
    for (x = 0; x <= amount / 5; x++) {         // 五角个数
        for (y = 0; y <= amount / 2; y++) {     // 两角个数
            z = amount - 5 * x - 2 * y;         // 剩余由一角补齐
            if (z >= 0) {
                printf("%d个五角 %d个两角 %d个一角\n", x, y, z);
                count++;
            }
        }
    }
    printf("共 %d 种凑法\n", count);
    return 0;
}
```

5. 求和

思路： 循环累加，注意 sum 初始化为 0。

```c
#include <stdio.h>

int main()
{
    int n, i, sum = 0;
    printf("请输入 n: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        sum += i;
    }
    printf("1 到 %d 的和是 %d\n", n, sum);
    return 0;
}
```

6. 整数的逆序输出

思路： 反复 `%10` 取个位、`/10` 去个位，把取出的位接到结果尾部。

```c
#include <stdio.h>

int main()
{
    int x, digit, ret = 0;
    printf("请输入一个整数: ");
    scanf("%d", &x);
    while (x > 0) {
        digit = x % 10;
        ret = ret * 10 + digit;
        x /= 10;
    }
    printf("逆序为 %d\n", ret);
    return 0;
}
```

7. 整数正序输出（逐位分解）

思路： 先构造最高位的 mask（10、100、…），再反复 `/mask` 取最高位、`%mask` 去掉最高位。

```c
#include <stdio.h>

int main()
{
    int x, mask = 1, t;
    printf("请输入一个整数: ");
    scanf("%d", &x);
```

```
    t = x;
    while (t > 9) {          // 构造最高位的 mask
        t /= 10;
        mask *= 10;
    }
    do {
        printf("%d", x / mask);
        if (mask > 9) printf(" ");
        x %= mask;
        mask /= 10;
    } while (mask > 0);
    return 0;
}
```

8. 求最大公约数

思路： 辗转相除法（欧几里得算法）：gcd(a, b) = gcd(b, a % b)，直到 b 为 0。

```
#include <stdio.h>

int main()
{
    int a, b, t;
    printf("请输入两个整数：");
    scanf("%d %d", &a, &b);
    while (b != 0) {
        t = a % b;
        a = b;
        b = t;
    }
    printf("最大公约数是 %d\n", a);
    return 0;
}
```

9. 水仙花数

思路： 三位数等于其各位数字立方和。分解出百位、十位、个位后判断。

```
#include <stdio.h>

int main()
{
    int i, a, b, c;
    for (i = 100; i <= 999; i++) {
        a = i / 100;         // 百位
        b = i / 10 % 10;     // 十位
        c = i % 10;          // 个位
        if (a*a*a + b*b*b + c*c*c == i) {
            printf("%d\n", i);
```

```
        }
    }
    return 0;
}
```

10. 99 乘法表

思路： 嵌套循环：外层控制行（1~9），内层控制列（1~行号）。

```c
#include <stdio.h>

int main()
{
    int i, j;
    for (i = 1; i <= 9; i++) {
        for (j = 1; j <= i; j++) {
            printf("%d×%d=%d\t", j, i, i * j);
        }
        printf("\n");
    }
    return 0;
}
```

11. 统计素数并求和

思路： 在 100~200 之间判断素数，是素数则计数并累加。判断时只需试除到 `j*j <= i`。

```c
#include <stdio.h>

int main()
{
    int i, j, isPrime, count = 0, sum = 0;
    for (i = 100; i <= 200; i++) {
        isPrime = 1;
        for (j = 2; j * j <= i; j++) {
            if (i % j == 0) {
                isPrime = 0;
                break;
            }
        }
        if (isPrime) {
            count++;
            sum += i;
        }
    }
    printf("素数个数: %d, 和: %d\n", count, sum);
    return 0;
}
```

12. 猜数字游戏（给出一个数字，让计算机猜）

思路： 人在心中想一个 1~100 的数，计算机用二分查找猜，人回答大了 / 小了 / 对了。

```c
#include <stdio.h>

int main()
{
    int low = 1, high = 100, mid;
    char ans;
    printf("请在心中想一个 1~100 的数，按回车开始\n");
    getchar();
    while (low <= high) {
        mid = (low + high) / 2;
        printf("我猜是 %d？(大L/小S/对Y)：", mid);
        scanf(" %c", &ans);
        if (ans == 'Y' || ans == 'y') {
            printf("我猜中啦！\n");
            return 0;
        } else if (ans == 'L' || ans == 'l') {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    printf("不可能！你骗我！\n");
    return 0;
}
```

13. 构造素数表

思路： 筛法（埃拉托斯特尼筛法）：把每个素数的倍数全部划掉，剩下的就是素数。

```c
#include <stdio.h>

int main()
{
    const int n = 100;
    int prime[n + 1];       // 0 表示是素数，1 表示被划掉
    int i, j;
    for (i = 0; i <= n; i++) {
        prime[i] = 0;
    }
    for (i = 2; i * i <= n; i++) {
        if (prime[i] == 0) {
            for (j = i * i; j <= n; j += i) {
                prime[j] = 1;
            }
        }
    }
```

```
    for (i = 2; i <= n; i++) {
        if (prime[i] == 0) {
            printf("%d ", i);
        }
    }
    return 0;
}
```

14. 井字棋

思路： 用 3×3 二维数组存棋盘，双方轮流落子，每次落子后检查横、竖、对角线是否连成一条线。

```
#include <stdio.h>

char board[3][3];   // ' ' 空, 'X', 'O'

void init(void)
{
    int i, j;
    for (i = 0; i < 3; i++)
        for (j = 0; j < 3; j++)
            board[i][j] = ' ';
}

void draw(void)
{
    printf(" %c | %c | %c \n", board[0][0], board[0][1], board[0][2]);
    printf("---+---+---\n");
    printf(" %c | %c | %c \n", board[1][0], board[1][1], board[1][2]);
    printf("---+---+---\n");
    printf(" %c | %c | %c \n", board[2][0], board[2][1], board[2][2]);
}

int win(char c)
{
    int i;
    for (i = 0; i < 3; i++) {
        if (board[i][0] == c && board[i][1] == c && board[i][2] == c) return 1;
        if (board[0][i] == c && board[1][i] == c && board[2][i] == c) return 1;
    }
    if (board[0][0] == c && board[1][1] == c && board[2][2] == c) return 1;
    if (board[0][2] == c && board[1][1] == c && board[2][0] == c) return 1;
    return 0;
}

int main(void)
{
    int x, y, turn = 0;
    char c;
    init();
```

```
    draw();
    while (turn < 9) {
        c = (turn % 2 == 0) ? 'X' : 'O';
        printf("%c 方落子，输入行列 (0-2 0-2)：", c);
        scanf("%d %d", &x, &y);
        if (x < 0 || x > 2 || y < 0 || y > 2 || board[x][y] != ' ') {
            printf("无效位置，重新输入\n");
            continue;
        }
        board[x][y] = c;
        draw();
        if (win(c)) {
            printf("%c 获胜！\n", c);
            return 0;
        }
        turn++;
    }
    printf("平局！\n");
    return 0;
}
```